

Python小练习：使用t-SNE算法将高维数据降维并可视化

作者：凯鲁嘎吉 - 博客园 <http://www.cnblogs.com/kailugaji/>

写一个Python小实例，使用t-SNE算法将高维数据降维到2维并可视化。主要功能是：生成带有3个类别的随机高维数据(50维)、使用t-SNE算法将其降维到2维、可视化降维结果，用不同颜色显示不同类别、保存可视化结果为图片文件。

Python代码

```
# Python代码：使用t-SNE算法将高维数据降维到2维并可视化
# 这段代码的主要功能是：
# 生成带有3个类别的随机高维数据(50维)
# 使用t-SNE算法将其降维到2维
# 可视化降维结果，用不同颜色显示不同类别
# 保存可视化结果为图片文件
# 作者：凯鲁嘎吉 - 博客园 http://www.cnblogs.com/kailugaji/

import numpy as np
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE # 从sklearn导入t-SNE算法

# 设置随机种子以便结果可复现
np.random.seed(42)

# 1. 生成随机高维数据 生成1000个样本，每个样本有50个特征的随机数据。
n_samples = 1000
n_features = 50
X_high_dim = np.random.randn(n_samples, n_features)

# 创建3个不同的类别，并根据类别对数据进行偏移，使不同类别的数据有所区分。
labels = np.random.randint(0, 3, n_samples)
for i in range(3):
    X_high_dim[labels == i] += i * 2

print(f"生成的高维数据形状：{X_high_dim.shape}") # 打印高维数据的形状(1000, 50)。

# 2. 使用t-SNE降维到2维
# 初始化t-SNE模型，设置：
# 降维到2维
# 困惑度为30(控制局部和全局结构的平衡)
# 最大迭代300次
# 随机种子42
tsne = TSNE(n_components=2, # 降维到2维
            perplexity=30, # 通常建议值在5-50之间
            max_iter=300, # 使用max_iter替代n_iter
            random_state=42)

X_low_dim = tsne.fit_transform(X_high_dim)
print(f"降维后的数据形状：{X_low_dim.shape}")
# 对高维数据执行t-SNE降维，并打印降维后的形状(1000, 2)。

# 3. 绘制二维t-SNE图
plt.figure(figsize=(10, 8)) # 创建10x8英寸的画布
scatter = plt.scatter(X_low_dim[:, 0], X_low_dim[:, 1], c=labels,
                    cmap='viridis', alpha=0.7, s=30)
```

```

# 创建散点图：
# x,y轴为降维后的两个维度
# 颜色根据标签区分
# 使用viridis颜色映射
# 透明度0.7
# 点大小30

# 添加图例和标题
plt.legend(*scatter.legend_elements(), title="Classes") # 添加图例
plt.title('t-SNE Visualization of High-dimensional Random Data') # 标题
plt.xlabel('t-SNE dimension 1') # x轴标签
plt.ylabel('t-SNE dimension 2') # y轴标签
plt.colorbar(scatter, label='Class') # 添加颜色条
plt.grid(True, alpha=0.3) # 添加半透明网格

# 保存图像到文件并显示
plt.savefig('tsne_visualization.png')
print("图像已保存")
plt.show()

```

可视化结果

